P3104R5
Bit permutations

Published Proposal, 2026-05-10

This version:
https://eisenwave.github.io/cpp-proposals/bit-permutations.html

Author:
Jan Schultke

Audience:
LWG

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Source:
eisenwave/cpp-proposals

Abstract

Add bit permutation functions to the bit manipulation library.

Table of Contents

1. Revision history

1.1. Changes since R4

- Fix a mismatch between synopsis comment and subclause title

1.2. Changes since R3

The paper was seen by LEWG at Sofia 2025, with the following outcome:

> **ACTION**: Ask SG6 to look at the paper and bring up issues / design questions back to LEWG if exists.
>
> **POLL**: Forward "P3104R4: Bit permutations" to LWG for C++29.
>
> | SF | F | N | SA | SA |
> |---|---|---|---|---|
> | **7** | 13 | 1 | 0 | 1 |

The following changes were made in preparation for the paper to be seen by SG6 and LWG:

- Mention lack of SIMD support in § 6.6 Why no SIMD support?
- Convert SVG images to equivalent MathML in § 8 Proposed wording (purely editorial change, for accessibility and aesthetics)
- Other minor editorial wording changes
- Fix typo in formula for `bit_repeat` (~~N~~ n) in § 8 Proposed wording

§ 1.3. Changes since R2

- Revise the proposed wording.
- Drop the `l` variants of `bit_compress` and `bit_expand`.

§ 1.4. Changes since R1

The paper has been seen in Tokyo by SG18 with positive reception, except for the `*_bit_permutation` functions. These have been removed to strengthen consensus.

§ 1.5. Changes since R0

- Expand § 4.3 Hardware support, taking more instructions into account, including AVX-512.
- Minor editorial fixes.

§ 2. Introduction

The C++ bit manipulation library in `<bit>` is an invaluable abstraction from hardware operations. Functions like `countl_zero` help the programmer avoid use of intrinsic functions or inline assembly.

However, there are still a few operations which are non-trivial to implement in software and have widely available hardware support. Therefore, I propose to expand the bit manipulation library with multiple bit permutation functions described below. Even without hardware support, these functions provide great utility and/or help the developer write more expressive code.

§ 3. Proposed features

I propose to add the following functions to the bit manipulation library (`<bit>`).

> NOTE: All constraints are exposition-only. The function bodies contain a naive implementation that merely illustrates the behavior.

> NOTE: See § 4.3 Hardware support for the corresponding hardware instructions.

§ 3.1. `bit_reverse`

```
template<unsigned_integral T>
constexpr T bit_reverse(T x) noexcept
{
    T result = 0;
    for (int i = 0; i < numeric_limits<T>::digits; ++i) {
        result <<= 1;
        result |= x & 1;
        x >>= 1;
    }
    return result;
}
```

Reverses the bits of `x` so that the least significant bit becomes the most significant.

> EXAMPLE 1 ¶
> `bit_reverse(uint32_t{0x00001234})` equals `0x24c80000`.

§ 3.2. `bit_repeat`

```
template<unsigned_integral T>
constexpr T bit_repeat(T x, int length) noexcept(false)
{
    T result = 0;
    for (int i = 0; i < numeric_limits<T>::digits; ++i) {
        result |= ((x >> (i % length)) & 1) << i;
    }
    return result;
}
```

Repeats a pattern stored in the least significant `length` bits of `x`, as many times as fits into `x`.

> EXAMPLE 2 ¶
> `bit_repeat(uint32_t{0xc}, 4)` equals `0xcccccccc`.

§ 3.3. `bit_compress`

```
template<unsigned_integral T>
constexpr T bit_compress(T x, T m) noexcept
{
    T result = 0;
    for (int i = 0, j = 0; i < numeric_limits<T>::digits; ++i) {
        bool mask_bit = (m >> i) & 1;
        result |= (mask_bit & (x >> i)) << j;
        j += mask_bit;
    }
    return result;
}
```

For each one-bit in `m`, the corresponding bit in `x` is taken and packed contiguously into the result, starting with the least significant result bit.

> EXAMPLE 3 ¶
>
> ```
> uint32_t x =          /* ... */;  // a b c d
> uint32_t m =             0b0101;  // 0 1 0 1
> uint32_t z = bit_compress(x, m);  // 0 0 b d
> ```
>
> Notice that bits `a` and `c` in `x` are ignored because the corresponding bit in the "mask" `m` is `0`. The remaining bits `b` and `d` are tightly packed to the right.

§ 3.4. `bit_expand`

```
template<unsigned_integral T>
constexpr T bit_expand(T x, T m) noexcept
{
    T result = 0;
    for (int i = 0, j = 0; i < numeric_limits<T>::digits; ++i) {
        bool mask_bit = (m >> i) & 1;
        result |= (mask_bit & (x >> j)) << i;
        j += mask_bit;
```

```
    }
    return result;
}
```

For each one-bit in `m`, a bit from `x`, starting with the least significant bit is taken and shifted into the corresponding position of the `m` bit.

EXAMPLE 4 ¶

```
uint32_t x =        /* ... */;  // a b c d
uint32_t m =           0b0101;  // 0 1 0 1
uint32_t z = bit_expand(x, m);  // 0 c 0 d
```

Notice that the bits `a` and `b` in `x` are ignored because the "mask" `m` only has two one-bits. The remaining bits `c` and `d` are placed where `m` has a one-bit.

§ 4. Motivation and scope

Bit-reversal, repetition, compression, and expansion are fundamental operations that meet multiple criteria which make them suitable for standardization:

1. They are common and useful operations.
2. They can be used to implement numerous other operations.
3. At least on some architectures, they have direct hardware support.
4. They are non-trivial to implement efficiently in software.
5. For known masks, numerous optimization opportunities are available.

§ 4.1. Applications

§ 4.1.1. Applications of `bit_reverse`

Bit-reversal is a common operation with uses in:

- **Cryptography**: scrambling bits
- **Networking**: as part of cyclic redundancy check computation

- **Graphics**: mirroring of images with one bit per pixel
- **Random number generation**: reversal can counteract low entropy of low-order bits such as in the case of linear congruential generators
- **Digital signal processing**: for radix-2 Cooley-Tukey FFT algorithms
- **Code obfuscation**: security by obscurity

§ 4.1.2. Applications of `bit_repeat`

The generation of recurring bit patterns is such a fundamental operation that it's hard to tie to any specific domain, but here are some use cases:

- **Debugging**: using obvious recurring bit pattenrs like `0xcccc...` to identify "garbage memory"
- **Bit manipulation**: generating alternating sequences of `1` and `0` for various algorithms
- **Eliminating integer divison**: for `x >> (i % N)` with very small `N`, `bit_repeat(x, N) >> i` can be used instead
- **Testing**: recurring bit patterns can make for good test cases when implementing numerics
- **Error detection**: known bit patterns can be introduced to spot failed transmission

§ 4.1.3. Applications of `bit_compress` and `bit_expand`

Compression and expansion are also common, with uses in:

- **Space-filling curves**: Morton/Z-Order and Hilbert curves
- **Input/output**: especially for variable-length encodings, such as UTF-8 (§ 4.2.3 UTF-8 decoding with bit_compress)
- **Chess engines**: for bitboards; see [ChessProgramming1]
- **Genomics**: according to [ARM1]

A GitHub code search for `/(_pdep_u|_pext_u)(32|64)/ AND language:c++` reveals ~1300 files which use the intrinsic wrappers for the x86 instructions.

§ 4.2. Motivating examples

§ 4.2.1. Implementing `countr_zero` with `bit_repeat`

[Anderson1] contains a vast amount of algorithms, many of which involve masks of alternating `0`s and `1`s.

When written as "magic numbers" in code, these masks can make it quite hard to understand the overall pattern and to generalize these algorithms. `bit_repeat` allows one to be more expressive here:

EXAMPLE 5 ¶

```
unsigned int v;      // 32-bit word input to count zero bits on right
unsigned int c = 32; // c will be the number of zero bits on the right
v &= -v;
if (v) c--;
if (v & 0x0000FFFF) bit_repeat((1 << 16) - 1, 32)) c -= 16;
if (v & 0x00FF00FF) bit_repeat( (1 << 8) - 1, 16)) c -=  8;
if (v & 0x0F0F0F0F) bit_repeat( (1 << 4) - 1,  8)) c -=  4;
if (v & 0x33333333) bit_repeat( (1 << 2) - 1,  4)) c -=  2;
if (v & 0x55555555) bit_repeat( (1 << 1) - 1,  2)) c -=  1;
```

It is now obvious how this can be expressed in a loop:

```
// ...
for (int i = 16; i != 0; i /= 2) {
    unsigned mask = bit_repeat((1u << i) - 1, i * 2);
    if (v & mask) c -= i;
}
```

`bit_repeat` has been an invaluable asset in the implementation of the remaining functions in this proposal (see [Schultke1] for details).

§ 4.2.2. Interleaving bits with `bit_expand`

A common use case for expansion is interleaving bits. This translates Cartesian coordinates to the index on a Z-order curve. Space filling curves are a popular technique in compression.

EXAMPLE 6

```
unsigned x = 3; // 0b011
unsigned y = 5; // 0b101
const auto i = bit_expand(x, bit_repeat(0b10u, 2)) // i = 0b01'10'11
             | bit_expand(y, bit_repeat(0b01u, 2));
```

4.2.3. UTF-8 decoding with `bit_compress`

`bit_compress` and `bit_expand` are useful in various I/O-related applications. They are particularly helpful for dealing with variable-length encodings, where "data bits" are interrupted by bits which signal continuation of the data.

EXAMPLE 7

The following code reads 4 bytes from some source of UTF-8 data and returns the codepoint. For the sake of simplicity, let's ignore details like reaching the end of the file, or I/O errors.

```
uint_least32_t x = load32_little_endian(utf8_data_pointer);
switch (countl_one(uint8_t(x))) {
case 0: return bit_compress(x, 0b01111111);
case 1: /* error */;
case 2: return bit_compress(x, 0b00111111'00011111);
case 3: return bit_compress(x, 0b00111111'00111111'00001111);
case 4: return bit_compress(x, 0b00111111'00111111'00111111'00000111);
}
```

4.2.4. Other operations based on `bit_compress` and `bit_expand`

Many operations can be built on top of `bit_compress` and `bit_expand`. However, direct hardware support is often needed for the proposed functions to efficiently implement them. Even without such support, they can be canonalized into a faster form. The point is that `bit_compress` and `bit_expand` allow you to *express* these operations.

EXAMPLE 8 ¶

```
// x & 0xf
bit_expand(x, 0xf)
bit_compress(x, 0xf)

// (x & 0xf) << 4
bit_expand(x, 0xf0)
// (x >> 4) & 0xf
bit_compress(x, 0xf0)

// Clear the least significant one-bit of x.
x ^= bit_expand(1, x)
// Clear the nth least significant one-bit of x.
x ^= bit_expand(1 << n, x)
// Clear the n least significant one-bits of x.
x ^= bit_expand((1 << n) - 1, x)

// (x >> n) & 1
bit_compress(x, 1 << n)
// Get the least significant bit of x.
bit_compress(x, x) & 1
// Get the nth least significant bit of x.
(bit_compress(x, x) >> n) & 1

// popcount(x)
countr_one(bit_compress(-1u, x))
countr_one(bit_compress(x, x))
```

§ 4.3. Hardware support

Operation	x86_64	ARM	RISC-V
bit_reverse	vpshufbitqmb[AVX512_BITALG], (bswap)	rbit[SVE2]	rev8[Zbb]+brev8[Zbkb], (rev8[Zbb])
bit_repeat	vpshufbitqmb[AVX512_BITALG]		
bit_compress	pext[BMI2]	bext[SVE2]	(vcompress[V])
bit_expand	pdep[BMI2]	bdep[SVE2]	(viota+vrgather[V])

(Parenthesized) entries signal that the instruction does not directly implement the function, but greatly assists in its implementation.

> NOTE: The AVX-512 `vpshufbitqmb` instruction can implement any bit permutation in the mathematical sense, and more.

> NOTE: The RISC-V `brev8` instruction can also be found under the name `rev.b`. There appears to have been a name change in 2022.

4.3.1. Support for `bit_reverse`

This operation is directly implemented in ARM through `rbit`.

Any architecture with support for `byteswap` (such as x86 with `bswap`) also supports bit-reversal in part. [Warren1] presents an O(log n) algorithm which operates by swapping lower and upper `N / 2`, ..., `16`, `8`, `4`, `2`, and `1` bits in parallel. Byte-swapping implements these individual swaps up to 8 bits, requiring only three more parallel swaps in software:

```
// assuming a byte is an octet of bits, and assuming the width of x is a po
x = byteswap(x);
x = (x & 0x0F0F0F0F) <<  4 | (x & 0xF0F0F0F0) >>  4; // ... quartets of bit
x = (x & 0x33333333) <<  2 | (x & 0xCCCCCCCC) >>  2; // ... pairs of bits
x = (x & 0x55555555) <<  1 | (x & 0xAAAAAAAA) >>  1; // ... individual bits
```

It is worth noting that clang provides a cross-platform family of intrinsics. `__builtin_bitreverse` uses byte-swapping or bit-reversal instructions if possible.

Such an intrinsic has been requested from GCC users a number of times in [GNU1].

4.3.2. Support for `bit_repeat`

Firstly, note that for the pattern length, there are only up to `N` relevant cases, where `N` is the operand width in bits. It is feasible to `switch` between these cases, where the length is constant in each case.

While the AVX-512 instruction `vpshufbitqmb` can be used for *all* cases, this is not the ideal solution for most cases. For very low or very great lengths, a naive solution is sufficient (and even optimal), where we simply use `<<` and `|` to duplicate the pattern. What actually matters is how often the pattern is repeated, i.e. `N / length`.

Specific cases like `bit_repeat(x, 8)`, `bit_repeat(x, 16)` can be implemented using permutation/duplication/gather/broadcast instructions.

However, note that the primary use of `bit_repeat` is to express repeating bit patterns without magic numbers, i.e. to improve code quality. Often, both the pattern and the length are known at compile-time, making hardware support less relevant. Even without hardware support, the reference implementation [Schultke1] requires only O(log N) fundamental bitwise operations.

§ 4.3.3. Support for `bit_compress` and `bit_expand`

Starting with Haswell (2013), Intel CPUs directly implement compression and expansion with with `pext` and `pdep` respectively. AMD CPUs starting with Zen 3 implement `pext` and `pdep` with 3 cycles latency, like Intel. Zen 2 and older implement `pext`/`pdep` in microcode, with 18 cycles latency.

ARM also supports these operations directly with `bext`, `bdep`, and `bgrp` in the SVE2 instruction set. [Warren1] mentions other older architectures with direct support.

Overall, only recent instruction set extensions offer this functionality directly. However, when the mask is a constant, many different strategies for hardware acceleration open up. For example

- interleaving bits can be assisted (though not fully implemented) using ARM `zip1`/`zip2`
- other permutations can be assisted by ARM `tbl` and `tbx`

As [Warren1] explains, the cost of computing `bit_compress` and `bit_expand` in software is dramatically lower for a constant mask. For specific known masks (such as a mask with a single one-bit), the cost is extremely low.

All in all, there are multiple factors that strongly suggest a standard library implementation:

1. The strategy for computing `bit_compress` and `bit_expand` depends greatly on the architecture and on information about the mask, even if the exact mask isn't known.
    - `tzcnt`, `clmul` (see [Schultke1] or [Zp7] for specifics), and `popcnt` are helpful.
2. ISO C++ does not offer a mechanism through which all of this information can be utilized. Namely, it is not possible to change strategy based on information that only becomes available during optimization passes. Compiler extensions such as `__builtin_constant_p` offer a workaround.
3. ISO C++ does not offer a mechanism through which function implementations can be chosen based on the surrounding context. In a situation where multiple `bit_compress` calls with the same mask `m` are performed, it is significantly faster to pre-compute information based on the

mask once, and utilize it in subsequent calls. The same technique can be used to accelerate integer division for multiple divisions with the same divisor.

Bullets 2. and 3. suggest that `bit_compress` and `bit_expand` benefit from being implemented directly in the compiler via intrinsic, even if hardware does not directly implement these operations.

Even with a complete lack of hardware support, a software implementation of `compress_bitsr` in [Schultke1] emits essentially optimal code if the mask is known.

EXAMPLE 9 ¶

```
unsigned bit_compress_known_mask(unsigned x) {
    return cxx26bp::bit_compress(x, 0xf0f0u);
}
```

Clang 18 emits the following (and GCC virtually the same); see [CompilerExplorer1]:

```
bit_compress_known_mask(unsigned int): # bit_compress_known_mask(unsigne
        mov     eax, edi               # {   unsigned int eax = edi;
        shr     eax, 4                 #     eax >>= 4;
        and     eax, 15                #     eax &= 0xf;
        shr     edi, 8                 #     edi >>= 8;
        and     edi, 240               #     edi &= 0xf0;
        or      eax, edi               #     eax |= edi;
        ret                            #     return eax; }
```

Knowing the implementation of `bit_compress`, this feels like dark magic. This is an optimizing compiler at its finest hour.

§ 5. Impact on existing code

This proposal is purely a standard library expansion. No existing code is affected.

§ 6. Design considerations

The design choices in this paper are based on [P0553R4], wherever applicable.

§ 6.1. Signature of `bit_repeat`

`bit_repeat` follows the "use `int` if possible" rule mentioned in [P0553R4]. Other functions such as `std::rotl` and `std::rotr` also accept an `int`.

It is also the only function not marked `noexcept`. It does not throw, but it is not `noexcept` due to its narrow contract (Lakos rule).

§ 6.2. Why the names *compress* and *expand*?

The use of `compress` and `expand` is consistent with the mask-based permutations for `std::simd` proposed in [P2664R6].

Furthermore, there are multiple synonymous sets of terminology:

1. `deposit` and `extract`
2. `compress` and `expand`
3. `gather` and `scatter`

I have decided against `deposit` and `extract` because of its ambiguity:

Taking the input `0b10101` and densely packing it to `0b111` could be described as:

> Extract each second bit from `0b10101` and densely deposit it into the result.

Similarly, taking the input `0b111` and expanding it into `0b10101` could be described as:

> Extract each bit from `0b111` and sparsely deposit it in the result.

Both operations can be described with `extract` and `deposit` terminology, making it virtually useless for keeping the operations apart. `gather` and `scatter` are simply the least common way to describe these operations, which makes `compress` and `expand` the best candidates.

Further design choices are consistent with [P0553R4]. The abbreviations `l` and `r` for left/right are consistent with `rotl`/`rotr`. The prefix `bit_` is consistent with `bit_floor` and `bit_ceil`.

6.3. Why the lack of generalization?

6.3.1. No generalized `bit_compress` and `bit_expand`

[N3864] originally suggested much more general versions of compression and expansion, which support:

1. performing the operation not just on the whole operand, but on "words" of it, in parallel
2. performing the operation not just on bits, but on arbitrarily sized groups of bits

I don't propose this generality for the following reasons:

1. The utility functions in `<bit>` are not meant to provide a full bitwise manipulation library, but fundamental operations, especially those that can be accelerated in hardware while still having reasonable software fallbacks.
2. These more general form can be built on top of the proposed hardware-oriented versions. This can be done with relative ease and with little to no overhead.
3. The generality falsely suggests hardware support for all forms, despite the function only being accelerated for specific inputs. This makes the performance characteristics unpredictable.
4. The proposed functions have wide contracts and can be `noexcept` (Lakos rule). Adding additional parameters would likely require a narrow contract.
5. Generality adds complexity to the standardization process, to implementation, and from the perspective of language users. It is unclear whether this added complexity is worth it in this case.

6.3.2. No generalized `bit_reverse`

Bit reversal can also be generalized to work with any group size:

```
template <typename T>
T bit_reverse(T x, int group_size = 1) noexcept(false);
```

With this generalization, `byteswap(x)` on conventional platforms is equivalent to `bit_reverse(x, 8)`.

However, this general version is much less used, not as consistently supported in hardware, and has a narrow contract. `group_size` must be a nonzero factor of `x` for this operation to be meaningful.

Therefore, a generalized bit-reversal is not proposed in this paper.

§ 6.4. Why does the signature of `bit_compress` require two same `T`s?

Initially, I went through a number of different signatures.

```
template<unsigned_integral T, unsigned_integral X>
constexpr T bit_compress(X x, T m) noexcept;
```

This signature is quite clever because the result never has more bits than the mask `m`. However, the the behavior is not immediately obvious when working zero-extension occurs, especially when the mask has more one-bits than `x` is wide.

Since this proposal includes low-level bit operations, it is reasonable and safe to require the user to be explicit. A call to `bit_compress` or `bit_expand` with two different types is likely a design flaw or bug. Therefore, I have settled on the very simple signature:

```
template<unsigned_integral T>
constexpr T bit_compress(T x, T m) noexcept;
```

> NOTE: The proposal originally included a variant of `bit_compress` which compresses to the left (to the most significant bits), and mixed widths would be even more complicated for that variant.

§ 6.5. Why no left variant for `bit_compress`?

The paper originally had `bit_compressl` and `bit_expandl` counterparts which are biased towards the most significant bits rather than the least significant bits. These have been removed because

- they add clunkyness compared to just having one function,
- [P2664R6] doesn't have left/right variants either, and we want symmetry with `std::simd`,
- users will want the right-hand variants almost every time anyway, and
- only the right-hand variants have direct hardware support in the form of `pext`, `pdep` et al.

§ 6.6. Why no SIMD support?

Following [P2933R4], almost all `<bit>` functions also have SIMD overloads. This means that the current design introduces inconsistency.

However, R3 of this proposal is already design-approved by LEWG, so to avoid holding up this paper's progress based on `std::simd` overloads, those are proposed separately, in [P3772R0].

7. Possible implementation

7.1. Reference implementation

All proposed functions have been implemented in [Schultke1]. This reference implementation is compatible with all three major compilers, and leverages hardware support from ARM and x86_64 where possible.

7.2. Other implementations

[Warren1] presents algorithms which are the basis for [Schultke1].

- An O(log n) `bit_reverse`
- An O($\log^2$ n) `bit_compress` and `bit_expand`
  - can be O(log n) with hardware support for carry-less multiplication aka. GF(2) polynomial multiplication

[Zp7] offers fast software implementations for `pext` and `pdep`, optimized for x86_64.

[StackOverflow1] contains discussion of various possible software implementations of `bit_compress` and `bit_expand`.

8. Proposed wording

The wording is relative to [N5008].

In subclause [version.syn], paragraph 2, update the synopsis as follows:

```
#define __cpp_lib_bitops                    201907L20XXXXL // freestandi
```

In subclause [bit.syn], update the synopsis as follows:

```
[…]
template<class T>
  constexpr int popcount(T x) noexcept;

// [bit.permute], permutation
template<class T>
  constexpr T bit_reverse(T x) noexcept;
template<class T>
  constexpr T bit_repeat(T x, int l);
template<class T>
  constexpr T bit_compress(T x, T m) noexcept;
template<class T>
  constexpr T bit_expand(T x, T m) noexcept;

// [bit.endian], endian
[…]
```

In subclause [bit], add the following subclause after [bit.count]:

X.X.X Permutation [bit.permute]

1 In the following descriptions, let N denote the value of `numeric_limits<T>::digits`, and let α_n denote the value of the n-th least significant bit in the base-2 representation of an integer α, so that α equals $\sum_{n=0}^{N-1} \alpha_n 2^n$.

```
template<class T>
  constexpr T bit_reverse(T x) noexcept;
```

2 *Constraints*: `T` is an unsigned integer type ([basic.fundamental]).

3 *Returns*: $\text{reverse}(x)$, where reverse is given by [FORMULA 1], and x is `x`.

$$\text{reverse}(x) = \sum_{n=0}^{N-1} x_n 2^{N-n-1} \qquad \text{[FORMULA 1]}$$

[*Note*: `bit_reverse(bit_reverse(x))` equals `x`. — *end note*]

```
template<class T>
  constexpr T bit_repeat(T x, int l);
```

4 *Constraints*: `T` is an unsigned integer type ([basic.fundamental]).

5 *Preconditions*: `l` is greater than zero.

6 *Returns*: $\text{repeat}(x, m)$, where repeat is given by [FORMULA 2], x is `x`, and l is `l`.

7 *Throws*: Nothing.

$$\text{repeat}(x, l) = \sum_{n=0}^{N-1} x_{(n \mod l)} 2^n \qquad \text{[FORMULA 2]}$$

```
template<class T>
  constexpr T bit_compress(T x, T m) noexcept;
```

8 *Constraints*: `T` is an unsigned integer type ([basic.fundamental]).

9 *Returns*: $\text{compress}(x, m)$, where compress is given by [FORMULA 3], x is `x`, and m is `m`.

$$\text{compress}(x, m) = \sum_{n=0}^{N-1} m_n x_n 2^{\left(\sum_{k=0}^{n-1} m_k\right)} \qquad \text{[FORMULA 3]}$$

```
template<class T>
  constexpr T bit_expand(T x, T m) noexcept;
```

10 *Constraints*: `T` is an unsigned integer type ([basic.fundamental]).

11 *Returns*: $\text{expand}(x, m)$, where expand is given by [FORMULA 4], x is `x`, and m is `m`.

$$\text{expand}(x, m) = \sum_{n=0}^{N-1} m_n x_{\left(\sum_{k=0}^{n-1} m_k\right)} 2^n \qquad \text{[FORMULA 4]}$$

In the subclause above, substitute the symbolic placeholders [FORMULA *N*] for formula numbers, in the style of subclause [c.math].

NOTE: I would have preferred a less mathematical approach to defining these functions. However, it is too difficult to precisely define `bit_compress` and `bit_expand` without visual aids, pseudo-code, or other crutches.

§ 9. Acknowledgements

I greatly appreciate the assistance of Stack Overflow users in assisting me with research for this proposal. I especially thank Peter Cordes for his tireless and selfless dedication to sharing knowledge.

I also thank various Discord users from Together C & C++ and #include<C++> who have reviewed drafts of this proposal and shared their thoughts.

§ References

§ Normative References

[N5008]
Thomas Köppe. *Working Draft, Programming Languages — C++*. 15 March 2025. URL: https://wg21.link/n5008

§ Informative References

[Anderson1]
Sean Eron Anderson. *Bit Twiddling Hacks*. URL: https://graphics.stanford.edu/~seander/bithacks.html

[ARM1]

Arm Developer Community. *Introduction to SVE2, Issue 02, Revision 02*. URL: https://developer.arm.com/-/media/Arm%20Developer%20Community/PDF/102340_0001_02_en_introduction-to-sve2.pdf?revision=b208e56b-6569-4ae2-b6f3-cd7d5d1ecac3

[ChessProgramming1]

VA. *chessprogramming.org/BMI2, Applications*. URL: https://www.chessprogramming.org/BMI2#Applications

[CompilerExplorer1]

Jan Schultke. *Compiler Explorer example for bit_compress*. URL: https://godbolt.org/z/5dcTjE5x3

[GNU1]

Marc Glisse et al.. *Bug 50481 - builtin to reverse the bit order*. URL: https://gcc.gnu.org/bugzilla/show_bug.cgi?id=50481

[N3864]

Matthew Fioravante. *A constexpr bitwise operations library for C++*. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n3864.html

[P0553R4]

Jens Maurer. *Bit operations*. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p0553r4.html

[P2664R6]

Daniel Towner; Ruslan Arutyunyan. *Extend std::simd with permutation API*. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p2664r6.html#permute_by_mask

[P2933R4]

Daniel Towner, Ruslan Arutyunyan. *Extend <bit> header function with overloads for std::simd*. 17 February 2025. URL: https://wg21.link/p2933r4

[P3772R0]

Jan Schultke. *std::simd overloads for bit permutations*. URL: https://isocpp.org/files/papers/P3772R0.html

[Schultke1]

Jan Schultke. *C++26 Bit Permutations*. URL: https://github.com/Eisenwave/cxx26-bit-permutations

[StackOverflow1]

Jan Schultke et al.. *What is a fast fallback algorithm which emulates PDEP and PEXT in software?*. URL: https://stackoverflow.com/q/77834169/5740428

[Warren1]

Henry S. Warren, Jr.. *Hacker's Delight, 2nd Edition*.

[Zp7]
Zach Wegner. *Zach's Peppy Parallel-Prefix-Popcountin' PEXT/PDEP Polyfill*. URL: https://github.com/zwegner/zp7